Homework 2: Image Clasification

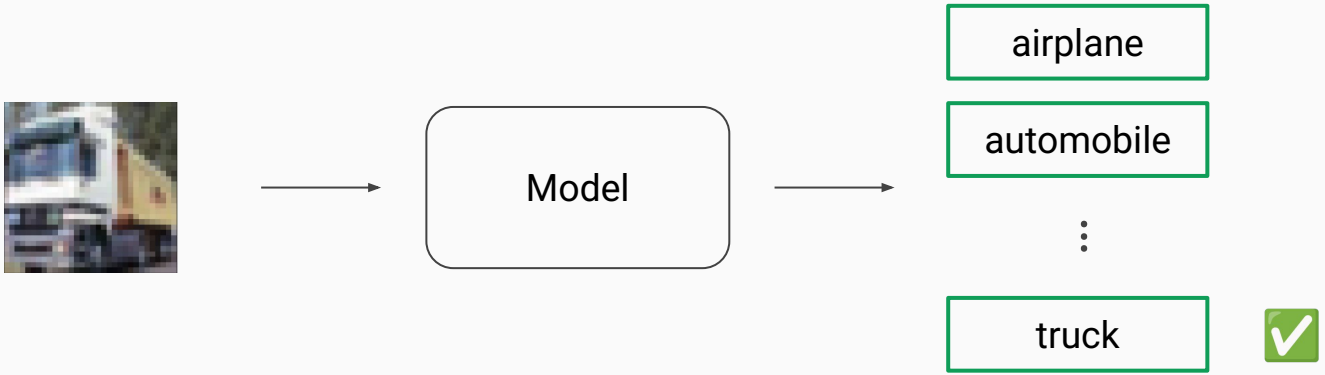
2026 深度學習 Deep Learning at Chang Gung University

Ying-Jia Lin

Date: 2026/10/14

Homework 2: Image Classification

- This homework targets image classification, which aims at classifying an image into one of the pre-defined categories.
- Multi-class classification (一筆資料只會有一個類別)

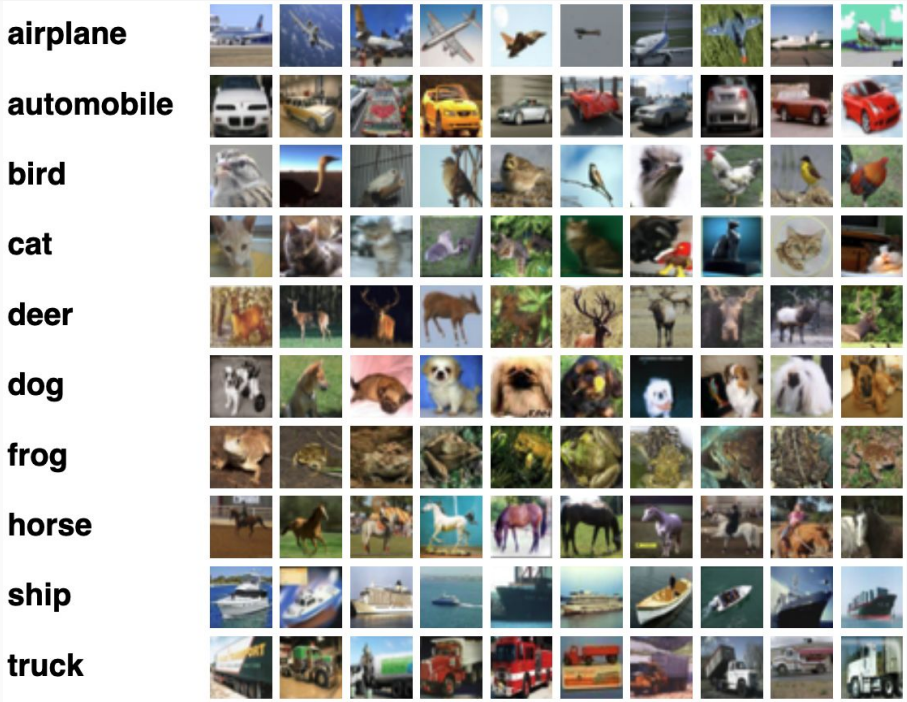


Dataset

CIFAR-10 (Canadian Institute for Advanced Research, 10 classes)

- Image size: 32 x 32
- RGB images
- Training set: 50,000 images
- Test set: 10,000 images
- **Class-balanced**
- **We don’t load data from torchvision this time!**

Dataset examples

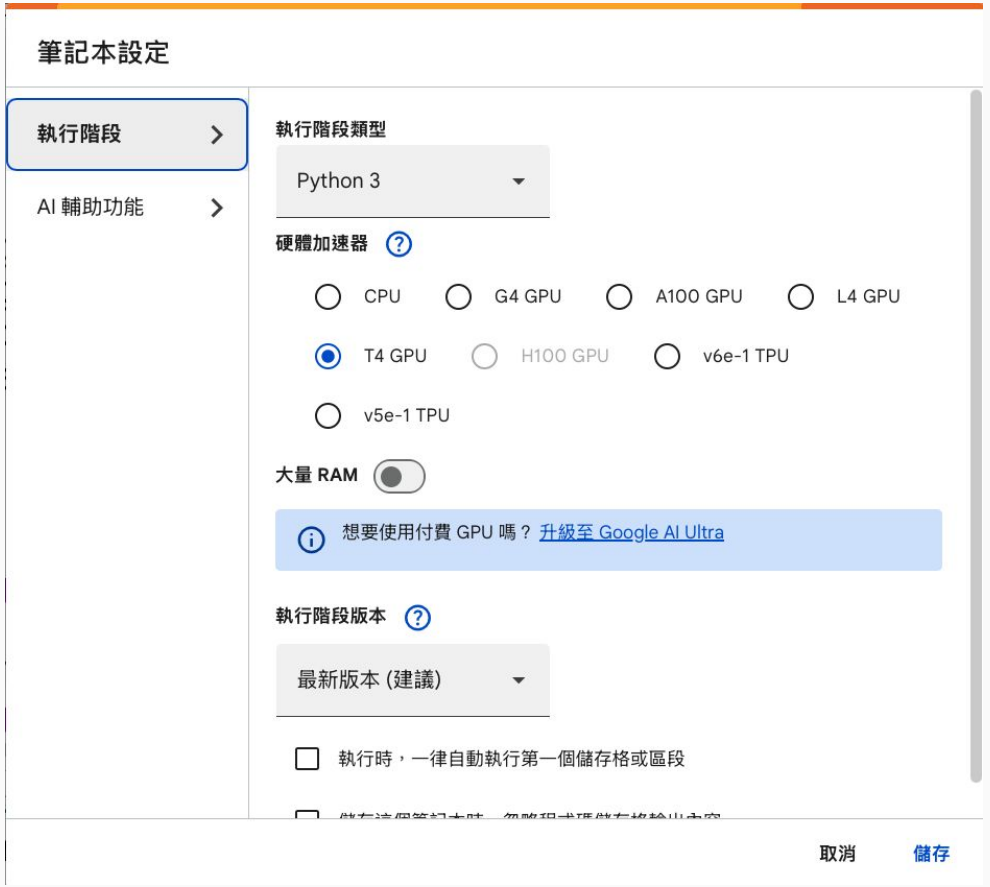


https://www.cs.toronto.edu/~kriz/cifar.html

Coding environment

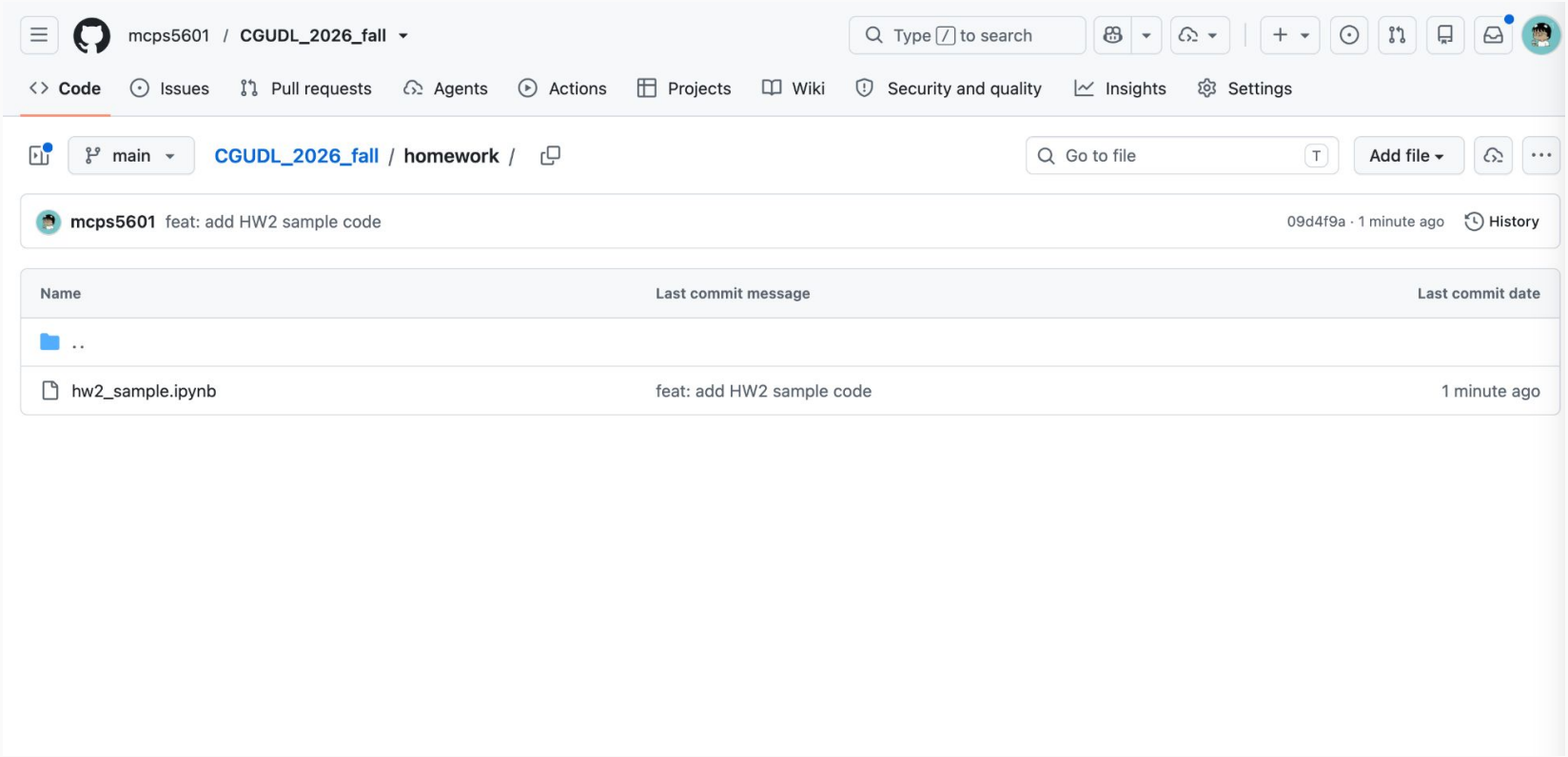
Google Colab 記得打開硬體加速器

路徑:(左上角) 編輯 -> 筆記本設定



Code (hints only)

Homework 2 material



https://github.com/mcps5601/CGUDL_2026_fall/blob/main/homework/hw2_sample.ipynb

Download dataset

下載 CIFAR-10 資料集

[58]
```
# 使用 wget 指令下載資料集
!wget https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
```

```
--2025-03-30 08:09:05--  https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
Resolving www.cs.toronto.edu (www.cs.toronto.edu)... 128.100.3.30
Connecting to www.cs.toronto.edu (www.cs.toronto.edu)|128.100.3.30|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 170498071 (163M) [application/x-gzip]
Saving to: 'cifar-10-python.tar.gz.1'

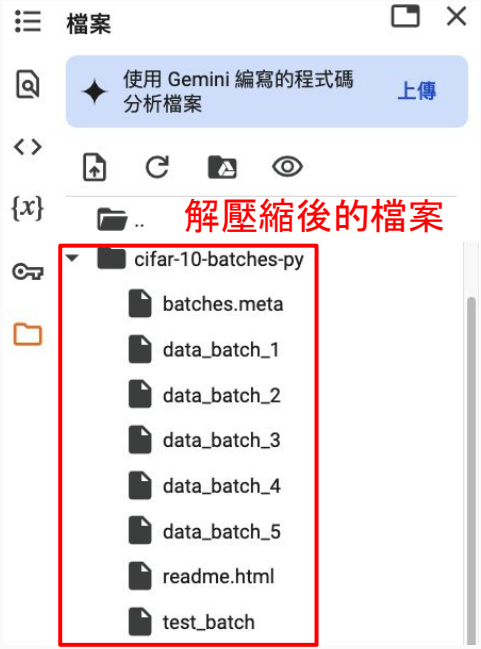
cifar-10-python.tar 100%[===================>] 162.60M  37.9MB/s    in 4.6s

2025-03-30 08:09:09 (35.4 MB/s) - 'cifar-10-python.tar.gz.1' saved [170498071/170498071]
```

[2]
```
# 使用 tar -xvf 進行解壓縮
# 可參考 https://note.drx.tw/2008/04/command.html
!tar -xvf cifar-10-python.tar.gz
```

```
cifar-10-batches-py/
cifar-10-batches-py/data_batch_4
cifar-10-batches-py/readme.html
cifar-10-batches-py/test_batch
cifar-10-batches-py/data_batch_3
cifar-10-batches-py/batches.meta
cifar-10-batches-py/data_batch_2
cifar-10-batches-py/data_batch_5
cifar-10-batches-py/data_batch_1
```

觀察解壓縮後的檔案




```python
# 觀察 CIFAR-10 的類別

with open("cifar-10-batches-py/batches.meta", "rb") as f:
    label_names = pickle.load(f, encoding='bytes')[b'label_names']
    label_names = [label.decode("utf-8") for label in label_names]

print(label_names)
```

```
['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
```

觀察解壓縮後的檔案內容

```python
def unpickle(file):
    with open(file, 'rb') as f:
        dict = pickle.load(f, encoding='bytes')
    return dict
```

```
[62]
```

```python
tmp = unpickle("cifar-10-batches-py/data_batch_1")
print(type(tmp))
print(tmp.keys())
```

```
<class 'dict'>
dict_keys([b'batch_label', b'labels', b'data', b'filenames'])
```

```
[66]
```

```python
print(tmp[b'data'].shape)
print(tmp[b'labels'])
print(tmp[b'data'][0])
print(tmp[b'data'][0].shape)
```

```
(10000, 3072)
[6, 9, 9, 4, 1, 1, 2, 7, 8, 3, 4, 7, 7, 2, 9, 9, 9, 3, 2, 6, 4, 3, 6, 6, 2, 6, 3, 5, 4, 0, 0,
[ 59  43  50 ... 140  84  72]
(3072,)
```

img_array →

原始資料是一維度的且按照RGB順序

```
[66]  1 print(tmp[b'data'].shape)
      2 print(tmp[b'labels'])
      3 print(tmp[b'data'][0])
      4 print(tmp[b'data'][0].shape)
```

```
(10000, 3072)
[6, 9, 9, 4, 1, 1, 2, 7, 8, 3, 4, 7, 7, 2, 9, 9, 9, 3, 2, 6, 4, 3, 6, 6, 2, 6, 3, 5, 4, 0, 0,
[ 59  43  50 ... 140  84  72]
(3072,)
```

img_array →

- 前 1024 個數值是 紅色通道 (R)
- 接下來 1024 個是 綠色通道 (G)
- 最後 1024 個是 藍色通道 (B)

TODO1: 資料 shape 轉換

```python
def save_images_from_batch(
    batch_data: np.ndarray,
    batch_labels: list,
    batch_idx: int,
    split: str,
    output_dir: str,
):
    for i in range(len(batch_data)):
        img_array = batch_data[i] # 每組 batch 的 shape: 張數 x 3096
        label = batch_labels[i]

        img = # TODO1: 將 img_array 轉成 3x32x32，再轉成 (Height, Width, Channel) 格式
        img = Image.fromarray(img) # 將數值資料轉為影像資料

        # 建立類別資料夾，讓影像可以存到各自的類別資料夾之中。
        class_dir = os.path.join(output_dir, split, str(label))
        os.makedirs(class_dir, exist_ok=True)
        # 因為這個 for loop 一次只會取一筆資料，因此需要 exist_ok=True
        # training 資料夾階層可以是 cifar10/train/`class_id`/train_n_0000.png
        # test 資料夾階層可以是 cifar10/test/`class_id`/test_n_0000.png

        img_filename = os.path.join(class_dir, f"{split}_{batch_idx}_{i:04d}.png")
        img.save(img_filename)
```

使用 reshape　使用 transpose

包裝前處理函式 (Wrapper)：convert_raw_to_images

```python
def convert_raw_to_images(cifar_dir: str = "cifar-10-batches-py", output_dir: str = "cifar10"):
    os.makedirs(output_dir, exist_ok=True)

    # 處理訓練集
    for i in range(1, 6): # data_batch 有 1 - 5
        batch = unpickle(os.path.join(cifar_dir, f"data_batch_{i}"))
        data = batch[b'data']
        labels = batch[b'labels']
        save_images_from_batch(
            batch_data=data,
            batch_labels=labels,
            batch_idx=i,
            split="train",
            output_dir=output_dir,
        )

    # 處理測試集
    test_batch = unpickle(os.path.join(cifar_dir, "test_batch"))
    save_images_from_batch(
        batch_data=test_batch[b'data'],
        batch_labels=test_batch[b'labels'],
        batch_idx=0,
        split="test",
        output_dir=output_dir,
    )
```

處理後的資料夾階層將如下所示

```
cifar10/
├── train/
│   ├── 0/
│   │   ├── train_0_0001.png
│   │   ├── train_0_0002.png
│   │   └── ...
│   ├── 1/
│   ├── 2/
│   ├── 3/
│   ├── 4/
│   ├── 5/
│   ├── 6/
│   ├── 7/
│   ├── 8/
│   └── 9/
│
└── test/
    ├── 0/
    ├── 1/
    ├── ...
    └── 9/
```

建立 PyTorch Dataset（code 範例介紹）

```
# 繼承 torch.utils.data.Dataset
class CIFAR10ImageFolder(torch.utils.data.Dataset):
    def __init__(self, root_dir: str, transform=None):
        """
        root_dir: 資料夾路徑，例如 'cifar10/train'
        transform: torchvision.transforms 用來轉換 PIL 圖片
        """
    return

    def __getitem__(self, idx: int):
        return

    def __len__(self):
        return
```

PyTorch Dataset (1): __init__

```python
class CIFAR10ImageFolder(torch.utils.data.Dataset):
    def __init__(self, root_dir: str, transform=None):
        """
        root_dir: 資料夾路徑，例如 'cifar10/train'
        transform: torchvision.transforms 用來轉換 PIL 圖片
        """
        self.root_dir = root_dir
        self.transform = transform
        self.image_paths = []
        self.labels = []

        # 列出 `root_dir` 底下有哪些類別資料夾，並由小至大排序
        class_names = sorted(os.listdir(root_dir))
        for idx, class_name in enumerate(class_names):
            class_folder = os.path.join(root_dir, class_name)
            if not os.path.isdir(class_folder):
                continue

            for filename in os.listdir(class_folder):
                if filename.endswith(('.png', '.jpg')):
                    # 將所有的檔案路徑都收集到 `self.image_paths`
                    self.image_paths.append(os.path.join(class_folder, filename))
                    # 將所有的 label_id 都收集到 `self.labels`
                    self.labels.append(idx)
```

收納盒

PyTorch Dataset (2, 3): __getitem__ & __len__

```
26  def __getitem__(self, idx: int):
27      img_path = self.image_paths[idx] # 取得 idx 所對應的檔案路徑（重要！！）
28      label = self.labels[idx]
29      image = Image.open(img_path).convert('RGB')  # 讀成 RGB 模式
30
31      if self.transform:
32          image = self.transform(image)
33
34      return image, label # 回傳影像資料與標籤
35
36  def __len__(self):
37      return len(self.image_paths)
```

執行建立 Dataset 的過程以及觀察類別數量

```
[ ]
1 # 定義資料數值轉換
2 transform = torchvision.transforms.Compose([
3     # transforms.RandomHorizontalFlip(),
4     torchvision.transforms.ToTensor(),  # (HWC, uint8) → (CHW, float32 [0,1])
5     torchvision.transforms.Normalize(mean=[0.5]*3, std=[0.5]*3)
6 ])
```

```
[ ]
1 # 建立 Dataset
2 train_dataset = CIFAR10ImageFolder(root_dir='cifar10/train', transform=transform)
3 test_dataset = CIFAR10ImageFolder(root_dir='cifar10/test', transform=transform)
```

```
[ ]
1 train_labels = [example[1] for example in train_dataset]
2 test_labels = [example[1] for example in test_dataset]
```

```
[ ]
1 import collections
2 print(collections.Counter(train_labels))
3 print(collections.Counter(test_labels))
```

```
Counter({0: 5000, 1: 5000, 2: 5000, 3: 5000, 4: 5000, 5: 5000, 6: 5000, 7: 5000, 8: 5000, 9: 5000})
Counter({0: 1000, 1: 1000, 2: 1000, 3: 1000, 4: 1000, 5: 1000, 6: 1000, 7: 1000, 8: 1000, 9: 1000})
```

從 train_dataset 切出 validation_dataset

```
[ ]
```

```python
# 從 train_dataset 切出 validation_dataset

from torch.utils.data import random_split

validation_ratio = 0.2
total_len = len(train_dataset)
val_size = int(validation_ratio * total_len)
train_size = total_len - val_size

train_dataset, val_dataset = random_split(train_dataset, [train_size, val_size])
```

TODO2: 建立 DataLoader

超參數

```
[ ]  1 LR = 0.001
     2 train_batch_size = 64
     3 test_batch_size = 128
     4 num_epochs = 10
```

```
 1 # TODO2: 建立 DataLoader (train / valid / test)
 2 train_loader = torch.utils.data.DataLoader(
 3     # Write your code here
 4 )
 5 val_loader = torch.utils.data.DataLoader(
 6     # Write your code here
 7 )
 8 test_loader = torch.utils.data.DataLoader(
 9     # Write your code here
10 )
11
12 # 簡單驗證一個 batch
13 for images, labels in test_loader:
14     print("Image shape:", images.shape)  # [B, 3, 32, 32]
15     print("Label shape:", labels.shape)  # [B]
16     break
```

```
Image shape: torch.Size([128, 3, 32, 32])
Label shape: torch.Size([128])
```

也可以用
next(iter(train_loader)
來檢查寫好的
DataLoader有沒有問題

建立 CNN

建立模型

- 影像在進行每層 convolution 後的 feature map 大小公式：

$$\text{output_size} = \frac{\text{input_size} - \text{kernel_size} + 2 * \text{padding}}{\text{stride}} + 1$$

```python
class SimpleCNN(torch.nn.Module):
    def __init__(self, num_classes=10, fc_hidden_size=256, dropout_ratio=0.5):
        super().__init__()

        self.conv1 = torch.nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1)
        self.conv2 = torch.nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = torch.nn.Conv2d(64, 128, kernel_size=3, padding=1)

        self.pool = torch.nn.MaxPool2d(kernel_size=2, stride=2) # 2x2 pooling

        self.fc1 = torch.nn.Linear(128 * 4 * 4, fc_hidden_size)
        self.fc2 = torch.nn.Linear(fc_hidden_size, num_classes)
        self.relu = torch.nn.ReLU() # 非線性轉換

        self.dropout = torch.nn.Dropout(dropout_ratio)
```

Convolutions (stride = 1)

Stride = 1

1	1	1	1	0	0
0	1	1	0	1	0
0	0	1	1	0	0
0	0	1	1	1	0
0	0	0	1	1	0
0	0	0	0	0	0

1	0	0
0	1	0
0	0	-1

Filter

1	1	1	2
-1	1	1	0
0	0	1	2
0	0	1	2

feature map

(6-3+0)/1+1=4



16

TODO3: 完成 CNN 模型的 forward funciton

```
17      def forward(self, x):
18          # TODO3: 加入 conv1 layer
19          # Write your code here
20          x = self.pool(x)
21          # conv1: output_size = (32 - 3 + 2 * 1) / 1 + 1 = 32
22          # pooling: output_size = (32 - 3 + 2 * 0) / 2 + 1 = 15.5 (進位為 16)
23
24          # TODO3: 加入 conv2 layer
25          # Write your code here
26          x = self.pool(x)
27          # conv2: output_size = (16 - 3 + 2 * 1) / 1 + 1 = 16
28          # pooling: output_size = (16 - 3 + 2 * 0) / 2 + 1 = 7.5 (進位為 8)
29
30          # TODO3: 加入 conv3 layer
31          # Write your code here
32          x = self.pool(x)    # shape: (B, 128, 4, 4)
33          # conv3: output_size = (8 - 3 + 2 * 1) / 1 + 1 = 8
34          # pooling: output_size = (8 - 3 + 2 * 0) / 2 + 1 = 3.5 (進位為 4)
35
36          x = x.view(x.size(0), -1)  # Flatten, shape: (B, 128*4*4)
37          x = # TODO3: 加入 fc1 layer
38          x = # TODO3: 加入 fc2 layer
39          return x
```

你的寫法不一定要跟我一樣

只要你的CNN可以順利運作💪

模型訓練前的準備

```
[ ]
1 # 設定設備
2 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
3 print(device)
```

```
cuda
```

```
[ ]
1 # 初始化模型、損失函數、optimizer
2 model = SimpleCNN().to(device)
3 criterion = torch.nn.CrossEntropyLoss()
4 optimizer = torch.optim.Adam(model.parameters(), lr=LR)
```

TODO4: 訓練（training）流程；TODO5: 驗證（validation）流程

```
train_losses, val_losses = [], []

for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    # TODO4: 請實作 training 流程

    # 驗證模型
    model.eval()
    val_loss = 0.0
    # TODO5: 請實作 validation 流程

    print(f'Epoch {epoch+1}/{num_epochs}, Train Loss: {train_losses[-1]:.4f}, '
      f'Val Loss: {val_losses[-1]:.4f}')

# 繪製 loss 的歷史記錄圖
plt.plot(range(1, num_epochs+1), train_losses, label='Train Loss')
plt.plot(range(1, num_epochs+1), val_losses, label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Training and Validation Loss')
plt.show()
```

驗證模型效能

```
model.eval()
predictions = []
test_loss = 0.0 # 在 test 的時候其實不用算 loss, 因為通常我們沒有 test set 的標準答案
test_losses = []

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        tmp_pred = outputs.argmax(dim=1) # tmp_pred 的型態是 tensor
        predictions.extend(tmp_pred.tolist()) # 如果用 append 的話會變成 [[1, 2, 3], [4, 5, 6]]

        loss = criterion(outputs, labels)
        test_loss += loss.item()
test_losses.append(test_loss / len(test_loader))
```

```
# test_labels = [example[1] for example in test_dataset]
```

```
accuracy_score(test_labels, predictions)
```

TODOs

#TODO1	將 img_array 轉成 3x32x32，再轉成 (Height, Width, Channel) 格式
#TODO2	建立 DataLoader (train / valid / test)
#TODO3	請完成模型 forward function
#TODO4	請實作 training 流程
#TODO5	請實作 validation 流程

Scoring

Coding work :

- 60%，其中 50% 來自於 TODOs，每個大項目 TODO 各佔 10%，剩下的 10% 來自於：
  - 你的模型達到基線分數 (Test accuracy >= 70%)：+3%
  - 你的模型達到 Test accuracy >= 75%：+2%
  - 你的模型達到 Test accuracy >= 80%：+5%
  - 以上分數只能使用 CNN 或 MLP 且禁止使用 pre-trained models（如 ResNet, VGG 等）

Report: 40%

- (10%) 請簡要說明你使用的模型架構（例如每層的Conv/FC/Pooling 數量、啟動函數、Dropout 等），即使是使用作業提供的baseline，也請寫出具體結構與設計考量。
- (5%) 請說明你模型在訓練與測試集上的準確率，並描述你做了哪些調整（如：資料前處理、模型設計、訓練策略等）來提升效能，這些改進帶來什麼變化？
- (10%) 錯誤圖片分析：請舉例說明你認為模型在哪些情況下會有較差表現，例如對哪些圖片、類別、或是訓練過程中遇到的問題等。
- (10%) 實驗設定分析：模型層數、optimizers、資料擴增與否，需有3個比較表格
- (5%) 自由發揮。若你有嘗試任意額外設計、ImageFolder、分析、畫圖、創意應用、實驗探索、或Test accuracy>=85% 挑戰者，請簡要說明。

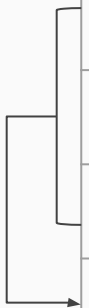
Delivery policies: File formats

- Coding work: Python notebook file (.ipynb)
  - 請直接於 Colab 上修改作業範例 .ipynb 檔案
- Package list: requirements.txt (optional)
  - E.g., torch==2.4.0
- Report: .docx (Word), no other formats allowed
- Zip the files above before uploading you homework.

Delivery policies: Filenames

任一命名錯誤/繳交格式錯誤，將作業總分 -10%

	Filename rule	**Filename example**
Coding work	DL_HW2_student_ID.ipynb	DL_HW2_12345678.ipynb
Report	DL_HW2_student_ID.docx	DL_HW2_12345678.docx
Package list	requirements.txt	
Zipped file	DL_HW2_student_ID.zip	DL_HW2_12345678.zip



Delivery policies: Things You should include

- In your report:

	Example	
Environment types	**If Colab or Kaggle**	**If local**
Running environment	Colab	System: Ubuntu 22.04, CPU: Ryzen 7-7800X3D
Python version	Colab	Python 3.12.1

Delivery policies: Rules of coding

- 如果你使用 ChatGPT 或其他生成式 AI 工具，請務必同時在以下兩個地方註明使用情況：
  - **程式碼註解（ipynb 中）**
  - **繳交的報告中**
- **嚴禁抄襲，不得直接複製、貼上其他同學的程式碼或作業 內容。**
- **如果參考網路上的程式碼，請提供參考來源的連結。**
- **以下行為將導致作業成績被扣分：**
  - (1) 使用生成式 AI 工具，但未註明使用情況。
  - (2) 參考網路資源，但未註明資料來源。
  - (3) 抄襲他人作品。

請上傳壓縮檔

- Please upload your file to E-Learning.
- You will have nearly four weeks to finish this assignment.
  - Deadline: 2026/11/10 23:59
- Feel free to ask questions.